Selecting Reliable LLM Reasoning by Testing Consensus Before Overturning It

DASE7506 Mini-project 2 · Final method: Stage 3 CREO · Seed: 20261022

Abstract

Given a maths question and four language-model solutions, we choose one candidate whose *final numeric answer* is correct. This is an answer-selection task: a correct final answer receives credit even if its written reasoning contains an error. Our research question is when to trust a repeated answer and when evidence from the candidate traces and their supplied embeddings justifies overturning it. We developed a NumPy-only sequence of increasingly structured selectors: plurality voting, a multi-positive linear candidate comparator, a listwise–pairwise hybrid, and **CREO** (Consensus Reliability and Evidence-Based Overturning). CREO scores parsed-answer groups, estimates consensus reliability, and gates between plurality and a learned challenger. Its frozen version achieves **755/798 (94.61%) on validation** and **387/467 (82.87%) on the complete official test bank**. The train-hard grouped out-of-fold (OOF) result is less favorable: **1720/2114 (81.36%)**, six fewer correct than the Stage 2 comparator. We report this disagreement, paired error strata, failed follow-up models, calibration limitations, and CPU/asset measurements rather than presenting the test score alone as evidence of a robust improvement.

1. Task, data, and research question

The supplied bank contains 4,755 training, 798 validation, and 467 test questions. Each question has four candidate traces and fixed 1024-dimensional embeddings, giving an array of shape [N, 4, 1024]. The official common.parse_answer converts a candidate's final answer to the comparison representation. We label candidate (j) positive when its parsed answer is non-null and equals the parsed reference answer. A question may have several positive candidates. The output is exactly one Python integer index in 0..3; no text generation is involved.

We define a **hard group** as a question with at least two distinct parseable candidate answers. All 467 test groups are hard, whereas only 2,114 training and 355 validation groups are hard. The large difference between validation-all and validation-hard accuracy therefore matters: validation-all contains many groups in which answer selection is less ambiguous. Our primary development comparison uses the same fixed five grouped folds on the 2,114 train-hard questions; all four candidates from one question stay in the same fold. Validation is a separate confirmation and selection set. Test references are used only for official final scoring and reproduction, never as model features or training labels.

The motivating hypothesis was that answer agreement is useful but can be systematically wrong. Plurality protects correct repeated answers; a comparator can rescue cases where a repeated answer misses a condition in the question. The practical question is whether a learned rule can gain enough correct overturns to offset the damage from overturning a correct consensus. We measured both sides of that trade-off, rather than treating a higher challenger score as proof of correct reasoning.

2. Method

2.1 Baselines and candidate comparators

The simplest baseline selects the largest parsed-answer cluster. Its tie rule uses a deterministic content hash of the question and candidate text, so it does not depend on question IDs or candidate position. Additional fixed-candidate, content-hash random, and longest-trace baselines measure whether training is needed at all. Unparseable candidates are masked during ordinary selection; the implementation has a deterministic fallback when all four are unparseable.

The Stage 1 comparator scores each candidate using 32 compact answer/consensus, surface-style, grounding, and conservative arithmetic controls, six relative embedding features, and the supplied embedding after subtracting the mean of the four candidates. The controls include answer support and vote margin, trace length and formatting, overlap with numbers in the question, and exact checks of a restricted set of simple arithmetic equalities. Failed parsing is treated as unknown, not as a mathematical error; arithmetic signals are soft features, not vetoes. The model is linear and trained with a **multi-positive listwise loss**:

L_i = logsumexp_{j in V_i}(s_ij) - logsumexp_{j in P_i}(s_ij),

where V_i contains parseable candidates and P_i is its subset of positive candidates. Separate L2 strengths regularize hand and embedding blocks; scalers are fitted only on each fold's training questions. Stage 2 adds a weighted pairwise term (weight 0.1) to the listwise objective. Its purpose is to sharpen comparisons on answer-diverse groups, not to add new external information.

2.2 CREO: answer groups and a reliability gate

Stage 3 first groups candidates with equivalent parsed answers. A learned **answer-group scorer** uses each group's support, plurality status, statistics of Stage 1/2 candidate scores, within- and between-group embedding relations, and summaries of grounding and conservative arithmetic features. It trains a linear group score with a listwise objective. Grouping ensures that two candidates giving the same answer contribute evidence to the same proposed answer. Stage 1 and Stage 2 models are frozen components inside the final checkpoint bundle.

A logistic reliability model then scores whether plurality is trustworthy from vote pattern, answer entropy, score differences, and group evidence. A second logistic gate compares plurality with the highest-scoring *non-plurality* answer group. The final Stage 3 configuration uses a learned-group challenger, reliability L2 0.01, balanced gate training with L2 0.1, and a fixed gate threshold of 0.6. The gate score is an operational decision signal; it is not asserted to be a calibrated probability that the answer is correct. When both branches choose the same answer group, there is no effective overturn. Candidate selection within the chosen group is deterministic. At inference, the method uses only the supplied question, four candidate texts, four embeddings, and the frozen checkpoint; neither IDs nor references enter the score.

3. Evaluation design and model selection

All learned models share seed 20261022 and the fixed question-level five folds. Training labels always use the official parser. The baseline and feature-block searches were bounded; Stage 1 used a predeclared simplicity preference when a larger model gained under 0.75 percentage point. Stage 2 advanced its pairwise/listwise hybrid because it gained about 0.99 point in train-hard CV and improved all five folds over Stage 1. We evaluated validation only after CV decisions, and saved the configurations, per-fold predictions, loss histories, and timing records under code/runs/.

Stage 3 compared answer-group scoring, fixed score pooling, and reliability gates. Its conservative replacement rule required nonnegative OOF change against Stage 2, at least four nondecreasing folds, at least two extra correct validation-hard questions, and no net harmful overturning. **No Stage 3 gate passed all these conditions.** The stable-direct gate was strongest on OOF but missed the validation-hard threshold by one question. The high-validation CREO gate improved validation-hard by eight questions but lost six OOF questions and improved only one fold. The high-validation gate was nevertheless explicitly chosen as the frozen final candidate *before its first official test*; this is a judgment based on validation evidence, not a claim that it met the robust replacement rule. The contradictory OOF evidence remains a central limitation.

We treated subsequent Stage 4 and Stage 5 experiments as diagnostic follow-ups. By then public test aggregates from earlier stages were known. Their test results are therefore labeled post-hoc or exploratory and were not used to train, tune, or choose the Stage 3 checkpoint reported here.

4. Results

4.1 Main comparison

Selector	Train-hard grouped OOF/CV	Validation all	Validation hard	Official full test
Parsed-answer plurality	1395/2114, 65.99%	86.22%	245/355, 69.01%	—
Stage 1 hand + centered embedding	1705/2114, 80.65%	749/798, 93.86%	306/355, 86.20%	—
Stage 2 listwise + pairwise	1726/2114, 81.65%	747/798, 93.61%	304/355, 85.63%	379/467, 81.16%
Stage 3 answer-group scorer	1721/2114, 81.41%	751/798, 94.11%	308/355, 86.76%	—
Stage 3 CREO, selected final method	**1720/2114, 81.36%**	**755/798, 94.61%**	**312/355, 87.89%**	**387/467, 82.87%**
Stage 3 stable-direct alternative	1736/2114, 82.12%	748/798, 93.73%	305/355, 85.92%	383/467, 82.01%

The dashes mean **not tested**, not zero accuracy. Stage 1 and Stage 2 train-hard five-fold standard errors are 0.85 and 1.02 percentage points, respectively. Stage 3 high-validation was frozen and hashed before its first official full-test evaluation. Its test accuracy is 1.71 percentage points (eight questions) higher than Stage 2's already-known score, but the test bank is small and this cross-stage difference is descriptive. At that first Stage 3 test, only the precommitted high-validation method was evaluated. The stable-direct alternative was also frozen and evaluated later, after the high-validation test aggregate was known; its 82.01% is a subsequent comparison, not a blind basis for changing the selected method.

4.2 Where CREO helps and hurts

Train-hard stratum, Stage 3 vs Stage 2	Groups	Both correct	Stage 2 only	CREO only	Both wrong
All hard	2114	1674	52	46	342
Wrong plurality	719	449	14	35	221
Correct plurality	1395	1225	38	11	121
Exactly one correct candidate	684	389	23	34	238

On train-hard groups, CREO recovers **21 net** wrong-plurality cases relative to Stage 2 and **11 net** single-correct cases. It loses **27 net** correct-plurality cases, leaving an overall six-question deficit. This directly identifies the failure mode: recognizing wrong consensus helps, but the gate does not protect correct consensus consistently across folds. CREO's fold changes against Stage 2 were [-2, -4, +6, -4, -2] correct questions.

Validation presents a different pattern. The frozen CREO gate solves 77/110 wrong-plurality hard groups versus 69/110 for Stage 2, while both solve 235/245 correct-plurality hard groups. CREO makes 98 effective validation overturns: 77 beneficial, 10 harmful, and 11 ineffective. Thus the gain of eight validation-hard questions has a concrete explanation, but the favorable balance does not repeat in OOF. The all-group validation accuracy of 94.61% is also easier to obtain than the hard-group 87.89%, because validation includes 443 non-hard questions. The all-hard test accuracy is 82.87%; this gap cautions against extrapolating validation-all to a new hard bank.

4.3 Ablations and negative extensions

Stage 1 structure-only features reached 74.41% train-hard CV, raw embeddings alone 78.38%, centered embeddings alone 78.90%, and the selected compact hand-plus-centered model 80.65%. Raw plus centered embeddings did not have a stable enough gain to justify the extra block. A uniform-positive cross-entropy ablation scored 80.75% ± 1.46 SE, only 0.09 point above the default listwise loss and with greater fold variation. The Stage 1 OOF plurality/comparator gate produced no effective improvement and was rejected. Stage 2's cluster pooling, lexical replacement, and targeted weighting also failed their advance criteria; its pairwise term was the useful change, though its validation accuracy fell by two questions relative to Stage 1.

Stage 4 tested whether an additional learned candidate and answer-group challenger would provide independent correction coverage. Its best three-way fixed-reliability rule scored 1700/2114 (80.41%) OOF and 737/798 (92.36%) validation-all, below Stage 2 and Stage 3. The compact interaction MLP and projected group models did not beat the frozen linear candidate comparator. A later user-authorized post-hoc test gave 379/467 (81.16%); it cannot be treated as blind selection evidence. The failure suggests that protecting consensus is insufficient if the proposed challenger loses too many single-correct and wrong-consensus cases.

Stage 5 added a **regime-conditioned residual revision** model over the Stage 3 anchor. On train-hard OOF, plurality and Stage 2 together offered 173 potentially correctable anchor errors, but selecting safe revisions was much harder: the retained 32-feature model made only six revisions, four beneficial, one harmful, and one neutral, improving 1720 to 1723 correct. It made **zero** revisions on validation; therefore it failed its predeclared validation promotion rule and the canonical choice retained Stage 3. At the user's request, a separately frozen residual-enabled variant was evaluated exploratorily after prior test exposure. It changed zero of 467 test choices and also scored 387/467. The equal score is a reproduction of the anchor's decisions, not an additional gain.

5. Uncertainty, interpretation, and limitations

The methods' softmax and logistic outputs are model scores, not guaranteed probabilities of correctness. Stage 1's OOF-fitted temperature did not calibrate validation well (five-bin ECE 0.3034; NLL 0.5641), so no confidence threshold based on that number was promoted. Stage 3's reliability score is used to rank and gate proposed answers; its OOF/validation NLL gap (0.35548 versus 0.28523 for the selected reliability setting) warns that the score distribution changes between splits. Stage 5's temperature reduced its residual model's OOF multiclass NLL (0.26201 to 0.25729), but it did not induce useful validation revisions. We report these diagnostics without equating a model's top score with correctness.

Several limits affect the conclusion. First, the fixed candidate bank contains at least one correct answer per question and the scorer checks only the final answer; the system does not establish whether an explanation is logically sound. Second, short traces, malformed numeric answers, and expressions outside the deliberately narrow arithmetic parser can evade verification. Third, the selected Stage 3 gate's OOF weakness is substantive evidence against a claim of uniformly superior generalization. Fourth, validation was observed during development and public test aggregates were later seen multiple

times; later Stage 4/5 test comparisons are non-blind. A fresh audit bank would provide the strongest check of whether the observed 82.87% carries over. We did not construct ID lookups, use fixed bank lengths, train on test references, add external data or pretrained weights, or generate extra LLM solutions.

6. Reproduction, resources, and integrity

The selected final method is the **Stage 3 CREO high-validation gate**, not the Stage 4 model or the Stage 5 residual-enabled exploratory variant. The top-level code/submission.py imports the unchanged predict function from code/submission_stage3.py. The latter and its four feature/model dependencies are byte-identical to the files recorded in the Stage 3 freeze manifest. The matching public inference bundle is submission_assets/checkpoint; it includes the Stage 1 and Stage 2 base models, the group scorer, the reliability model, the gate, scalers, and configuration. The other published files are submission_assets/final_test_score.json, submission_assets/freeze_manifest.json, and submission_assets/freeze_manifest.sha256. Their sources remain unchanged under code/runs/stage3/final_high_validation/ and code/runs/stage3/final_test_score.json. code/PACKAGE_MANIFEST.json records the original course release, **not** this final submission; SUBMISSION_MANIFEST.json records hashes and sizes for the current entry, dependencies, report, and public assets. Public directory URL template: https://github.com/<OWNER>/<REPO>/tree/<COMMIT>/submission_assets/.

Install Python 3.12 and the pinned numpy==2.5.3 dependency. Run the following from code/ in PowerShell (use the equivalent environment syntax on other systems). The prediction and scoring commands reproduce validation without retraining; substitute test and a distinct output name for independent official test reproduction.

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
$env:OMP_NUM_THREADS = '4'
$env:OPENBLAS_NUM_THREADS = '4'
$env:MKL_NUM_THREADS = '4'
$py = '.\.venv\Scripts\python.exe'
$ckpt = '..\submission_assets\checkpoint'
$pred = '.\runs\reproduced_validation_predictions.json'
$score = '.\runs\reproduced_validation_score.json'
$predictArgs = @(
  '.\predict.py', '--method', '.\submission.py',
  '--checkpoint', $ckpt, '--split', 'validation',
  '--output', $pred
)
& $py @predictArgs
$scoreArgs = @(
  '.\score_predictions.py', '--split', 'validation',
  '--predictions', $pred, '--output', $score
)
& $py @scoreArgs
```

To regenerate the training artifacts in a **fresh copy** of the project, run the following in order after installation and the four-thread settings above. The copy step gives Stage 3 the Stage 2 checkpoint path used in the original experiment; the copied Stage 2 bundle was verified byte-for-byte against the original frozen Stage 2 base. Keep the recorded configuration/selection rule rather than selecting a new model from the regenerated results.

```
& $py .\run_experiments.py
& $py .\run_stage2.py
New-Item -ItemType Directory -Force .\runs\final
$stage2 = '.\runs\stage2\provisional_model_bundle'
$base = '.\runs\final\frozen_checkpoint'
```

```
Copy-Item -LiteralPath $stage2 -Destination $base -Recurse
& $py .\run_stage3.py
```

The grouped folds are saved in code/runs/folds.json. Stage 3 training consumes saved Stage 1/2 OOF predictions and those base checkpoints, then writes its selected provisional bundle. The frozen checkpoint permits scoring **without retraining**. STAGE3_REPRODUCTION_NOTES.md and the stage summaries provide experiment commands and decision records. The updated top-level entry was checked against frozen pretest validation predictions: all 798 IDs and selected indices matched, and the official scorer again returned 755/798. All 31 current unit/regression tests pass; all 18 frozen Stage 3 file hashes still match the freeze manifest. This entry check did **not** run test again.

The Stage 3 checkpoint is **166,594 bytes uncompressed**, well below the 16 MiB inference-asset limit. On the local Windows 11 machine (Python 3.12.3, NumPy 2.5.3, AMD64 Family 23 Model 96, 16 logical CPUs), four-thread fresh-process validation predictions took 2.88–3.18 s in five recorded runs. The frozen full-test run took **2.775 s externally for prediction plus independent scoring** (official inner prediction 1.844 s, scoring 0.036 s), below the 30 s limit locally; the course reference CPU is the final eligibility authority. A validation-only in-process probe measured about 207 MiB peak working set, including Python, NumPy, input embeddings, extracted features, and models. The bounded Stage 1 search took 167 s, the complete Stage 2 search/audit phase 973 s, and the Stage 3 search 135 s on this machine; later Stage 4/5 follow-ups took approximately 503 s and 38 s, respectively, for their recorded canonical runs. The freeze manifest identifies every frozen source/asset hash, while the official test score JSON records the claimed 82.869379%.

7. Attribution and AI assistance

The supplied questions and references derive from GSM8K under its retained MIT licence; candidate solutions and the fixed 1024-dimensional embeddings were prepared with Qwen3-0.6B, as recorded in the project data manifest. We used only these provided materials and did not add external datasets, pretrained weights, or generated solutions. Course ideas used as motivation include embeddings (Lecture 1, PDF-reader pp. 38–40), similarity and softmax (Lecture 2, pp. 13–21), train/validation separation and overfitting (Lecture 3, pp. 30–42), logits and cross-entropy (Lecture 4, pp. 12–18), verifier framing and answer extraction (Lecture 5, pp. 40–59), self-consistency (Lecture 5, pp. 62–68; Lecture 6, pp. 18–23), scorer confidence versus correctness (Lecture 6, pp. 27–43), and calibration and proper scoring rules (Lecture 7, pp. 42–63). These lectures motivated design choices; they do not prove our empirical results.

AI coding-assistance disclosure: OpenAI Codex assisted substantively with the following: (1) research planning and experiment design; (2) feature and loss-function design; (3) Python implementation and debugging; (4) experiment orchestration and audit scripts; (5) result analysis; and (6) drafting and editing this report. All reported numbers are taken from saved local artifacts and official scoring outputs, not from the assistant's memory. The final method, resource claims, and interpretation were checked by the author against the linked frozen artifacts, including the Stage 3 freeze manifest, the official evaluation JSON, and the submission manifest. Reviewers should verify the reported claims against those artifacts rather than against this prose alone.